A Contention-Aware Duplication Heuristic for DAG Scheduling on NoC Systems

Barış Büyükyılmaz* and Oğulcan Uğuroğlu†
Department of Computer Engineering, Hacettepe University, Ankara, Türkiye
E-mails: *barisbuyukyilmaz@gmail.com, †ogulcan.uguroglu7@gmail.com

***Abstract*—Task scheduling of Directed Acyclic Graphs (DAGs) on multiprocessor systems is a well-known optimization problem in parallel and embedded computing. Task duplication is a commonly used technique to reduce inter-processor communication by replicating selected tasks across multiple processors [1]–[3].**

However, most classical duplication-based approaches rely on simplified communication models and ignore network contention. In modern Network-on-Chip (NoC) based systems, communication delays are strongly affected by shared network resources, routing conflicts, and bandwidth limitations [4]. As a result, duplication decisions that are beneficial under idealized models may lead to increased congestion and degraded performance in realistic systems.

In this work, we present CA-D, a contention-aware recursive ancestor duplication heuristic for DAG scheduling on 2D mesh NoC architectures. CA-D uses explicit per-link interval reservation during scheduling, evaluates all predecessors for duplication under the contention model, and applies a conservative redundant duplicate pruning pass. To enable fair comparison across schedulers that use different communication models, we introduce a contention replay evaluation methodology that re-evaluates all schedules under a common NoC model.

We compare CA-D against HEFT and a classical duplication baseline (CD-LS) on two experiment grids: a random DAG grid (72 instances) and a structured graph family grid (324 instances) covering six DAG topologies. CA-D achieves a mean replayed speedup of $1.47\times$ over HEFT on random DAGs and $2.52\times$ on structured graph families, consistently outperforming the contention-blind duplication baseline under fair evaluation.

***Index Terms*—task scheduling, task duplication, Network-on-Chip, contention-aware scheduling, DAG scheduling, NoC, MPSoC.**

I. Introduction

Task scheduling of Directed Acyclic Graphs (DAGs) on multiprocessor systems is a fundamental problem in parallel and embedded computing. In this problem, tasks are represented as nodes and dependencies as edges, and the objective is typically to minimize the overall execution time (makespan) while satisfying precedence and resource constraints.

Task duplication is a well-established technique used to reduce communication overhead in such systems. By duplicating selected parent tasks across processors, communication can be avoided or reduced, allowing successor tasks to start earlier. Classical duplication-based scheduling algorithms such as DBUS [1], HCPFD [2], and HLD [3] have demonstrated that duplication can significantly improve scheduling performance, particularly in communication-intensive workloads.

Despite their effectiveness, these approaches rely on simplified communication models that assume constant communication cost or contention-free networks. Such assumptions are not realistic for modern multiprocessor systems, especially Network-on-Chip (NoC) based architectures, where communication resources are shared and contention plays a critical role in determining performance.

Contention-aware scheduling has been proposed to address this limitation by explicitly modeling communication conflicts and network behavior. In particular, Sinnen et al. [4] introduced a contention-aware scheduling model that incorporates communication resource constraints into the scheduling process by scheduling communication edges on network links. This work shows that considering contention leads to more accurate and efficient schedules.

Under contention-aware models, task duplication becomes even more important, since reducing inter-processor communication can also reduce network contention. However, duplication decisions must be made carefully. Excessive or poorly placed duplication may increase resource usage and may not lead to performance improvements.

Motivated by this observation, this project investigates a contention-aware task duplication heuristic for DAG scheduling on NoC systems. The key idea is that duplication decisions should not be based solely on execution-time improvement, but should also consider their impact on network contention.

The proposed approach builds on a list-based scheduling framework similar to HEFT-like methods [5], and augments it with a contention-aware duplication heuristic. For each duplication candidate, the method evaluates both the reduction in earliest finish time and the associated communication cost under contention.

Unlike optimization-based approaches such as [6], which formulate duplication scheduling as an Integer Linear Programming (ILP) problem, this work focuses on a heuristic-based approach that is simpler to implement and suitable for practical scheduling scenarios.

This paper presents a contention-aware duplication strategy for DAG scheduling on NoC systems and evaluates its effectiveness under different workload and network conditions.

The main contributions of this work are as follows:

- A Python-based simulation framework for evaluating DAG scheduling heuristics on 2D mesh NoC systems with explicit per-link interval reservation.
- The CA-D scheduler, which combines contention-aware link reservation with greedy recursive ancestor duplication and conservative redundant duplicate pruning.

- A fair contention replay methodology that re-evaluates all schedules under a common NoC model, enabling valid comparison across schedulers with different communication models.
- Experimental evaluation on a random DAG grid (72 instances, varying edge density and CCR) demonstrating a mean replayed speedup of $1.47\times$ over HEFT.
- Experimental evaluation on six structured graph families (324 instances) demonstrating a mean replayed speedup of $2.52\times$ over HEFT, with topology-specific analysis.

II. Related Work

Task scheduling for Directed Acyclic Graphs (DAGs) has been extensively studied in multiprocessor and heterogeneous computing systems. Existing approaches can be categorized into duplication-based scheduling, resource-aware duplication strategies, and contention-aware scheduling methods.

A. Duplication-Based Scheduling

Duplication-based scheduling algorithms aim to reduce communication overhead by replicating tasks across multiple processors. Early works such as the *Duplication-Based Bottom-Up Scheduling* (DBUS) algorithm [1], the *Heterogeneous Critical Parent with Fast Duplication* (HCPFD) algorithm [2], and the *Heterogeneous Limited Duplication* (HLD) approach [3] exploit duplication to place parent tasks closer to dependent tasks, thereby reducing communication delays and improving makespan.

Subsequent works further refined duplication strategies by introducing selective duplication mechanisms. For example, improved duplication strategies reduce redundant task copies while maintaining performance [7]. Similarly, the *Heterogeneous Earliest Finish with Duplication* (HEFD) algorithm [5] extends list scheduling by incorporating duplication into heterogeneous environments.

Despite their effectiveness, these approaches rely on simplified communication models that assume constant communication costs and do not account for network contention.

B. Resource- and Energy-Aware Duplication

More recent approaches address the inefficiencies of aggressive duplication. Resource-aware scheduling methods aim to reduce redundant task replication while preserving scheduling performance [8]. Energy-aware duplication algorithms further consider the trade-off between performance and energy consumption [9].

These methods demonstrate that duplication must be carefully controlled, as excessive replication can degrade performance. However, they still do not consider network-level contention effects.

C. Contention-Aware Scheduling

Contention-aware scheduling approaches explicitly model communication conflicts and network behavior. Sinnen et al. [4] propose a contention-aware duplication scheduling algorithm that integrates communication contention into scheduling decisions. This work highlights the importance of modeling realistic communication behavior.

However, duplication decisions in such approaches are still primarily driven by scheduling heuristics and are not explicitly controlled based on network congestion.

D. NoC-Aware Scheduling Optimization

Tang et al. [6] extend scheduling models to Network-on-Chip (NoC) architectures and propose an optimization framework for duplication-based schedules. Their work considers both contention-free and contention-aware scenarios using Integer Linear Programming (ILP).

While this approach provides accurate modeling of NoC behavior, it assumes that duplication strategies are given and focuses on optimizing scheduling rather than improving duplication decisions.

E. Position of This Work

Although significant progress has been made in both duplication-based and contention-aware scheduling, the interaction between task duplication and network contention remains insufficiently explored. Existing methods either perform duplication without considering contention or consider contention without explicitly controlling duplication decisions.

This work addresses this gap by integrating network congestion directly into duplication decisions. Unlike prior approaches, duplication is explicitly regulated through a contention-aware decision rule, enabling more selective and efficient task replication.

A comparative summary of the discussed scheduling approaches is provided in Table I, highlighting key differences in duplication handling, contention awareness, and NoC modeling.

TABLE I
Comparison of scheduling approaches in terms of duplication, contention awareness, and NoC modeling capabilities

Method	Duplication	Contention-Aware	NoC Model	Resource-Aware
HEFT	✗	✗	✗	✗
DBUS / HCPFD / HLD	✓	✗	✗	✗
Bansal (Selective Dup.)	✓	✗	✗	✗
Mei et al.	✓	✗	✗	✓
Liang et al.	✓	✗	✗	✓
Sinnen et al.	✓	✓	✗	✗
Tang et al.	✗	✓	✓	✗
This Work	✓	✓	✓	✓

As shown in Table I, existing approaches do not simultaneously address duplication decision and network contention, which motivates the proposed method.

III. Problem Definition

This section formally defines the DAG scheduling problem under a contention-aware communication model with task duplication.

A. System Model

The application is represented as a Directed Acyclic Graph (DAG):

$$G = (V, E, w, c)$$

where:

- $V = \{v_1, v_2, \ldots, v_n\}$: set of tasks
- $E \subseteq V \times V$: precedence constraints
- w_i: computation cost of task v_i
- c_{ij}: communication volume from v_i to v_j

The target platform consists of processors:

$$P = \{p_1, p_2, \ldots, p_m\}$$

connected via a Network-on-Chip (NoC). Communication between processors is subject to routing delays and contention.

Tasks may be duplicated on multiple processors to reduce communication overhead.

B. Decision Variables

- $x_{ik} \in \{0, 1\}$: task v_i has an instance on processor p_k
- $y_{ik} \in \{0, 1\}$: processor p_k executes the primary instance of task v_i
- s_i^k: start time of task v_i on processor p_k, if an instance exists
- f_i^k: finish time of task v_i on processor p_k, if an instance exists

The processor assigned to the primary execution of task v_i is denoted by $proc(i)$, where $y_{ik} = 1$.

C. Objective

The objective is to minimize the makespan:

$$F = \max_{v_i \in V} f_i^{proc(i)}$$

D. Constraints

1) Primary Assignment Constraint: Each task has exactly one primary execution:

$$\sum_{k=1}^{m} y_{ik} = 1 \qquad \forall v_i \in V$$

2) Instance Existence Constraint: A primary assignment implies the existence of an instance:

$$y_{ik} \leq x_{ik} \qquad \forall v_i \in V,\ p_k \in P$$

3) Timing Constraint: For each existing instance:

$$f_i^k = s_i^k + w_i$$

4) Processor Constraint: Tasks assigned to the same processor cannot overlap in execution.

5) Precedence Constraint (Duplication-Aware): A task can start only after receiving data from all its parent tasks. For each parent, the earliest available data from any duplicated instance is considered:

$$s_j \geq \max_{v_i \in pred(v_j)} \left(\min_{k:\, x_{ik}=1} \left(f_i^k + D_{ij}^{eff}(k, proc(j)) \right) \right)$$

E. Communication Model

The communication duration for sending data with volume c_{ij} from processor p_k to processor p_l is:

$$D_{ij}^{base}(k, l) = \alpha \cdot hops_{kl} + \beta \cdot c_{ij} \tag{1}$$

where $hops_{kl}$ is the Manhattan distance under XY routing and α, β are per-hop latency and per-unit bandwidth coefficients. In our experiments, $\alpha = 0$ and $\beta = 1$, so duration equals communication volume.

Link-interval reservation. Each directed link L maintains a sorted list of occupied time intervals. When communication e_{ij} is scheduled from p_k to p_l, the XY route r_{kl} is determined and the earliest common slot where all links on the route are simultaneously free is found:

$$t_{start} = \textit{earliest\_route\_slot}(r_{kl},\ f_i^k,\ D_{ij}^{base}) \tag{2}$$

where f_i^k is the finish time of the source instance. The communication occupies the interval $[t_{start},\ t_{start} + D_{ij}^{base}]$ on every link along the route, and data becomes available at p_l at time $t_{start} + D_{ij}^{base}$.

Contention effect. If any link on the route is occupied at t_{start}, the communication is shifted right until all links are jointly free. The effective delay due to contention is:

$$D_{ij}^{eff}(k, l) = t_{start} + D_{ij}^{base} - f_i^k \ \geq\ D_{ij}^{base} \tag{3}$$

Contention emerges naturally from interval overlap rather than a parametric penalty function. Communications that share links with earlier reservations are delayed by the exact amount required to resolve the conflict. If $p_k = p_l$, no link is reserved and $D_{ij}^{eff} = 0$.

During duplication evaluation, the scheduling state is cloned so that tentative link reservations do not affect the committed schedule. This allows side-effect-free probing of candidate processor assignments.

F. Duplication Decision Rule

Duplication decisions are based on the improvement in earliest finish time (EFT):

$$\Delta EFT = EFT^{no\text{-}dup} - EFT^{dup}$$

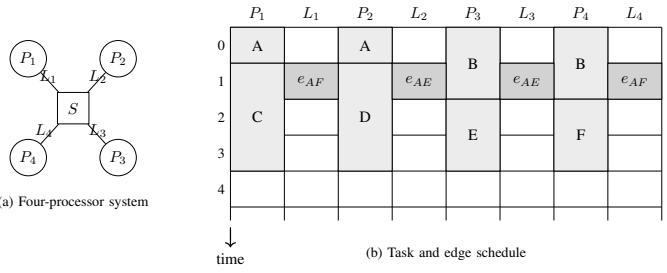
Duplication is applied if:

$$\Delta EFT > 0$$

G. Consistency Note

Duplication decisions are evaluated using tentative scheduling, where communication delays and contention effects are estimated before committing to duplication.

The effect of duplication under a contention-aware communication model is illustrated in Fig. 1. Duplicated instances of a task allow different communications to originate from different processors, which can reduce communication delays for successor tasks.



(a) Four-processor system

(b) Task and edge schedule

Fig. 1. Illustration of task duplication under the contention-aware model. Duplicated instances of task A allow different communications to be sent from different processors, reducing communication delay for successor tasks.

IV. System Architecture and Design

This section presents the overall system architecture and design of the proposed contention-aware duplication scheduling framework. The system is modeled as a scheduling pipeline operating on a DAG application and a Network-on-Chip (NoC) based multiprocessor platform.

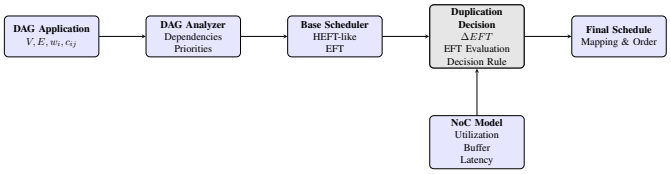


Fig. 2. System architecture of the proposed contention-aware duplication framework. A DAG application is analyzed and scheduled using a base HEFT-like scheduler. Candidate duplications are evaluated based on their impact on earliest finish time (EFT), where communication delays are estimated using a contention-aware NoC model.

A. Hardware/Software Architecture

The overall system operates on a multiprocessor platform connected through a Network-on-Chip (NoC).

The hardware architecture consists of:

- Multiple processing elements $P = \{p_1, p_2, \ldots, p_m\}$
- A 2D mesh NoC topology
- Routers and communication links supporting deterministic routing

On the software side, the system processes a DAG-based application, where tasks represent computation units and edges represent data dependencies.

The system includes the following components:

- **DAG Analyzer:** extracts dependencies and computes task priorities
- **Base Scheduler:** generates an initial schedule using a HEFT-like algorithm
- **Duplication Decision Module:** evaluates candidate duplications by estimating their impact on earliest finish time (EFT)
- **NoC Communication Model:** estimates communication delays and contention effects

The key contribution of this work lies in the duplication decision module, which uses a contention-aware communication model to guide duplication decisions indirectly through EFT evaluation.

B. Dataflow and System Model

The system follows a pipeline-based dataflow model, where scheduling decisions are performed in sequential stages.

The dataflow can be described as:

1) The input DAG is analyzed to extract task and communication parameters
2) The base scheduler generates an initial mapping using earliest finish time (EFT)
3) Candidate duplications are evaluated for each task
4) Duplication decisions are made based on the improvement in EFT
5) The final schedule is produced

Communication is explicitly modeled, and contention effects are incorporated through the NoC communication model.

C. Timing and Execution Model

The system follows a static scheduling model, where all scheduling decisions are made offline.

The execution model assumes:

- Non-preemptive task execution
- Deterministic computation times
- Communication delays dependent on routing and contention

A task can only begin execution after all its parent tasks have completed and the required data has been received.

D. Model of Computation

The system follows a dataflow model of computation:

- Nodes represent computational tasks
- Edges represent data dependencies
- Task execution is triggered by data availability

This model enables explicit handling of precedence constraints and communication effects.

E. Assumptions and Design Limitations

The proposed system is based on the following assumptions:

- The task graph is static and known in advance
- Execution times are deterministic
- Network topology is fixed (2D mesh)
- Routing is deterministic

The main limitations are:

- Dynamic or runtime scheduling is not considered
- Contention is estimated rather than measured at runtime
- Adaptive or iterative optimization is not included

F. Resource Model

The system assumes limited computational and communication resources:

- **Processing resources:** Each processor executes one task at a time
- **Communication bandwidth:** NoC links are shared and subject to contention
- **Memory usage:** Task duplication increases memory consumption

The proposed approach accounts for communication resource usage through the contention-aware communication model, which influences duplication decisions via EFT evaluation.

V. Methodology

This work improves task duplication decisions in DAG scheduling by incorporating contention-aware communication effects, implemented as explicit per-link interval reservation, into earliest finish time (EFT) evaluation. Three schedulers are compared: HEFT (baseline, no contention modeling, no duplication), CD-LS (classical duplication, analytic communication model), and the proposed CA-D (contention-aware link reservation with recursive ancestor duplication).

A. Scheduling Framework

All three schedulers share the same list-scheduling backbone:

1) Compute upward rank for each task and sort in descending order.
2) For each task v_j in priority order, evaluate all processors p_k and select the one with the minimum EFT.
3) Commit v_j to the selected processor.

The schedulers differ in how they model communication delays and whether they perform task duplication during processor evaluation.

B. HEFT Baseline

HEFT uses an analytic communication model: the delay of edge e_{ij} from p_k to p_l equals $D_{ij}^{base}(k,l)$ regardless of other communications. No link reservations are made and no task duplication is performed. HEFT serves as the reference baseline for speedup computation.

C. Classical Duplication Baseline (CD-LS)

CD-LS extends HEFT by evaluating whether duplicating any direct predecessor v_i of v_j onto the candidate processor p_k reduces EFT. The decision rule is:

$$\Delta EFT = EFT_j^{no\text{-}dup} - EFT_{j,k}^{dup} > 0$$

CD-LS uses the analytic communication model for this evaluation, so contention effects are not considered during duplication decisions.

D. CA-D: Contention-Aware Recursive Ancestor Duplication

CA-D uses the link-interval reservation model described in Section III-E during all EFT evaluations. For each candidate processor, the scheduling state is cloned so that tentative link reservations do not affect the committed schedule.

When a predecessor v_i is duplicated onto p_k, CA-D recursively evaluates all ancestors of v_i that are not already present on p_k. If placing an ancestor reduces the EFT of v_i on p_k by more than a threshold $\varepsilon \approx 10^{-12}$, the ancestor is also duplicated. Ancestors are evaluated in ascending task-id order.

The importance of selecting the appropriate source instance for communication is illustrated in Fig. 3. When multiple communications compete for the same network link, contention shifts the later communication right, increasing overall schedule length. CA-D explicitly models this effect during duplication evaluation.

1) Phase 15A: Greedy Recursive Ancestor Duplication:

```
Compute upward rank; sort tasks by descending rank
for each task v_j in priority order do
    for each processor p_k do
        S ← CLONE(schedule state)
        for each predecessor v_i of v_j not on p_k do
            EFT_0 ← EFT of v_j on p_k without duplicating v_i
            S' ← CLONE(S); place v_i on p_k in S'; RECURSIVEDUP(v_i, p_k, S')
            if EFT_0 − EFT in S' > ε then
                Commit v_i and ancestors to S
            end if
        end for
        EFT[p_k] ← EFT of v_j on p_k in S
    end for
    Commit v_j to processor p* = arg min_k EFT[p_k]
end for
```

2) Phase 15B: Conservative Redundant Duplicate Pruning: After all tasks are scheduled, a post-schedule pass removes duplicate instances that satisfy all four conditions simultaneously: (A) the instance is not the primary; (B) at least one other instance of the same task remains; (C) no committed communication uses it as the source; and (D) no successor on the same processor would lose its only local data source. Pruning does not reschedule tasks or reroute communications.

E. Fair Contention Replay

HEFT and CD-LS use the analytic communication model, so their native makespans do not reflect real NoC contention. CA-D uses link-interval reservation natively. Comparing native makespans directly mixes two different models and produces an invalid comparison.

To enable a fair comparison, we apply a *contention replay* pass to every schedule. The replay preserves the task-to-processor assignment and primary/duplicate flags from the original schedule, discards all timing, and recomputes start and finish times from scratch using the link-interval reservation model. The key metric is:

$$replayed\_speedup = \frac{HEFT_{\text{replayed}}}{S_{\text{replayed}}}$$

where both makespans are computed under the same contention model. CA-D and CD-LS schedules with *replay overhead ratio* ≈ 1.0 confirm that the contention model was already active during scheduling.

F. Deviations from the Extended Proposal

The extended proposal described a scalar contention penalty $CP_{ij}(k,l) = \lambda_1 U + \lambda_2 B + \lambda_3 L$ and single critical-parent duplication following Sinnen et al. [4]. The implemented system differs in three principal ways:

1) **Communication model:** The scalar penalty was replaced by explicit per-link interval reservation. This produces a replay overhead ratio near 1.0 for CA-D, confirming internal model consistency.
2) **Duplication scope:** Instead of selecting a single critical parent, CA-D evaluates all direct predecessors and recursively places their ancestors. This greedy approach is simpler and deterministic, though not globally optimal.
3) **Duplication selection order:** Ancestors are evaluated in ascending task-id order rather than by a global critical-path criterion.

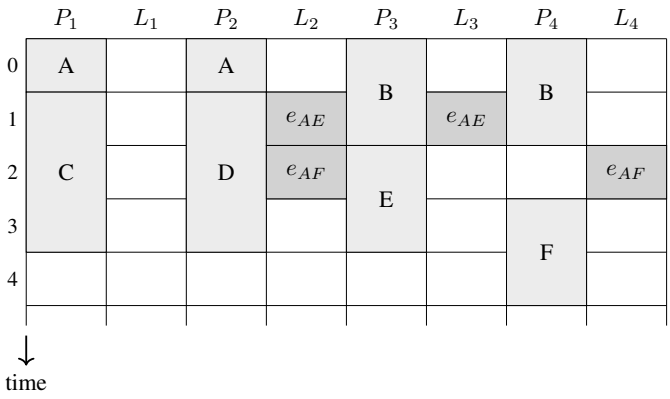


Fig. 3. Contention on one communication link delays e_{AF}, which causes task F to start later and increases the overall schedule length.

VI. Experimental Setup

This section describes the evaluation methodology used to assess the effectiveness of the proposed contention-aware duplication strategy.

A. Platform and Tools

Experiments are conducted using a custom Python simulation framework.

- **NoC Model:** 2D mesh topology, 4×4 (16 processors)
- **Routing Algorithm:** Deterministic XY routing
- **Communication Model:** Per-link interval reservation; $\alpha = 0$, $\beta = 1$
- **Simulation:** Python 3.10+; reproducible from fixed seeds via `configs/default.yaml`

B. Workloads and Experiments

Two experiment grids are evaluated.

Experiment A — Random DAG Grid. Random DAGs are generated using the Erdős-Rényi model.

- **Task count:** 20 (40-task runs excluded due to CA-D runtime; see Section VIII-C)
- **Edge probability:** $p \in \{0.25,\ 0.40\}$
- **CCR:** $\{0.1,\ 1.0,\ 5.0\}$
- **Seeds:** 0, 1, 2
- **Total instances:** 72 (3 seeds $\times$ 2 edge probabilities $\times$ 3 CCR)

Experiment B — Graph Family Grid. Six structured DAG families are evaluated to understand topology-specific scheduling behavior: fork, join, fork-join, in-tree, out-tree, and diamond. The chain family was excluded because its linear structure provides no parallelism for duplication to exploit. Representative topologies are shown in Fig. 4.

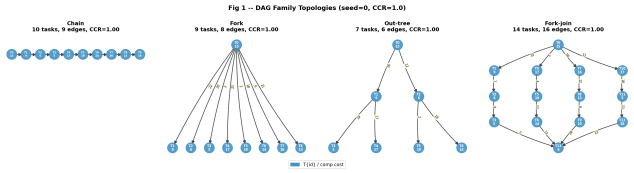


Fig. 4. DAG family topologies used in Experiment B (CCR = 1.0, seed = 0). From left to right: fork, join, out-tree, fork-join. Each topology exposes different communication and parallelism patterns.

- **Families:** fork, join, fork_join, in_tree, out_tree, diamond
- **Family configurations:** 9 parameter variants per family
- **CCR:** $\{0.1,\ 1.0,\ 5.0\}$
- **Seeds:** 0, 1, 2
- **Total instances:** 324 (6 families $\times$ 9 configs $\times$ 3 seeds $\times$ 3 CCR / 4 schedulers)

C. Evaluation Metrics

- **Makespan (F):** $\max_{v_i \in V} f_i^{proc(i)}$ — lower is better; reflects the scheduler's own model.
- **Replayed makespan:** Makespan recomputed via contention replay under the common link-reservation model. Provides a model-consistent comparison across all schedulers.
- **Replayed speedup:** $HEFT_{\text{replayed}}/S_{\text{replayed}}$ — primary fair comparison metric; values > 1 indicate improvement over the HEFT baseline.
- **Task instance ratio (TIR):** Total task instances / $|V|$ — quantifies duplication overhead; 1.0 = no duplication.
- **Replay overhead ratio:** $F_{\text{replayed}}/F_{\text{native}}$ — values near 1.0 confirm the scheduler's communication model is accurate.

D. Baselines

CA-D is compared against:

- **HEFT:** List scheduling without duplication; analytic communication model [5]. Serves as the reference baseline (speedup denominator).

- **CD-LS:** Parent-only duplication with analytic communication model [1], [2]. Isolates the effect of duplication without contention awareness.

E. Evaluation Procedure

For each workload instance, each scheduler produces a placement. All placements are then re-evaluated via contention replay. Mean metrics are reported over all seeds within each CCR and graph-family stratum.

VII. System-Level Considerations

This section discusses broader system-level aspects of the proposed approach. While the core contribution focuses on contention-aware duplication decisions, several system-level considerations are relevant for practical deployment.

A. Real-Time Constraints and Latency

The proposed method aims to reduce application makespan, which directly relates to end-to-end latency. By avoiding unnecessary duplication that may increase communication overhead, the method can improve latency in communication-intensive scenarios.

Implemented:

- Makespan optimization through EFT-based duplication decisions

Not implemented / future work:

- Explicit deadline-aware scheduling
- Worst-case execution time (WCET) analysis

B. Reliability and Fault Tolerance

Task duplication can inherently improve reliability by providing redundant execution paths. However, the proposed method uses duplication purely for performance optimization rather than fault tolerance.

Implemented:

- None (duplication is not used for reliability purposes)

Not implemented / future work:

- Fault-tolerant duplication strategies
- Failure-aware scheduling

C. Energy Efficiency

Duplication increases computational workload and may lead to higher energy consumption. By avoiding unnecessary duplication through EFT-based evaluation, the proposed method may indirectly reduce redundant computation, although energy efficiency is not explicitly optimized.

Implemented:

- Indirect reduction of redundant computation

Not implemented / future work:

- Explicit energy-aware optimization
- Power modeling and measurement

D. Security Considerations

The proposed method does not explicitly address security concerns. However, scheduling and communication patterns may influence system-level vulnerabilities.

Implemented:

- None

Not implemented / future work:

- Secure task mapping
- Communication isolation mechanisms

E. Scalability

The implemented system was evaluated on a 4×4 NoC (16 processors). Larger mesh sizes were not evaluated due to CA-D's high scheduling runtime at the current implementation complexity.

Implemented:

- Evaluation on 4×4 NoC with 20-task DAGs

Limitations and future work:

- Increased computational cost due to nested cloning during duplication evaluation (CA-D runtime is approximately $236\times$ HEFT on random DAGs)
- Evaluation on 8×8 and larger meshes is left as future work

F. Other Considerations

The proposed approach is designed for static scheduling scenarios and assumes full knowledge of the task graph and system parameters.

Limitations:

- Not suitable for dynamic or runtime scheduling
- Depends on accuracy of contention estimation

G. Resource Model

The system assumes limited computational and communication resources.

- **Processing resources:** Each processor executes one task at a time
- **Communication bandwidth:** NoC links are shared and subject to contention
- **Memory usage:** Task duplication increases memory consumption

The proposed approach accounts for communication resource usage through the contention-aware communication model, which influences scheduling decisions indirectly via earliest finish time (EFT).

VIII. Results and Evaluation

A. Performance Results

Table II summarizes the random DAG grid. The graph-family grid is reported separately in Table III after the CCR-level speedup trend.

Fig. 5 shows the replayed speedup as a function of CCR for each graph family. CA-D achieves the highest benefit at high CCR values, where communication-dominated workloads make link contention most significant.

TABLE II
MEAN RESULTS ON THE RANDOM DAG GRID (EXPERIMENT A). $n = 20$, 4×4 NOC, $\alpha = 0$, $\beta = 1$, 72 INSTANCES TOTAL.

Scheduler	Native Speedup	Replayed Speedup	TIR	Replay Overhead
HEFT	1.000	1.000 (ref)	1.000	1.436
CD-LS	1.055	1.078	1.306	1.397
CA-D	**1.050**	**1.474**	2.261	1.025

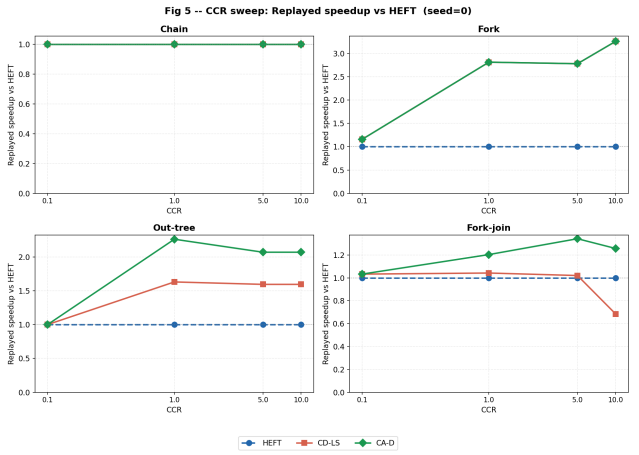


Fig. 5. Replayed speedup vs. CCR for HEFT, CD-LS, and CA-D across four DAG families (fork, join, out-tree, fork-join). Each panel shows mean results over 3 seeds. CA-D consistently achieves higher replayed speedup at high CCR.

TABLE III
MEAN RESULTS ON THE GRAPH FAMILY GRID (EXPERIMENT B). 6 FAMILIES, 4×4 NOC, 324 INSTANCES TOTAL.

Scheduler	Native Speedup	Replayed Speedup	TIR	Runtime (ms)
HEFT	1.000	1.000 (ref)	1.000	24.8
CD-LS	1.345	2.089	1.382	53.1
CA-D	**1.349**	**2.521**	1.651	341.2

Fig. 6 shows the task instance ratio (TIR) across CCR values. CA-D's higher TIR reflects recursive ancestor duplication, which places more copies than CD-LS's parent-only strategy but avoids unnecessary copies through Phase 15B pruning.

Fig. 7 illustrates an example CA-D schedule for an out-tree DAG at CCR = 1.0, showing how ancestor duplication eliminates remote communications and reduces the critical path.

B. Comparative Analysis

Fig. 8 illustrates why native makespan comparison is misleading. HEFT and CD-LS use the analytic communication model; their native makespans underestimate real execution time. Contention replay reveals the actual cost under the shared-link model.

Fig. 9 shows the replay overhead ratio ($F_{\text{replayed}}/F_{\text{native}}$) across CCR values. HEFT and CD-LS show overhead > 1 at medium and high CCR, confirming the optimism of their analytic models. CA-D's overhead remains near 1.0, confirming that link-interval reservation during scheduling is consistent with the physical contention model.

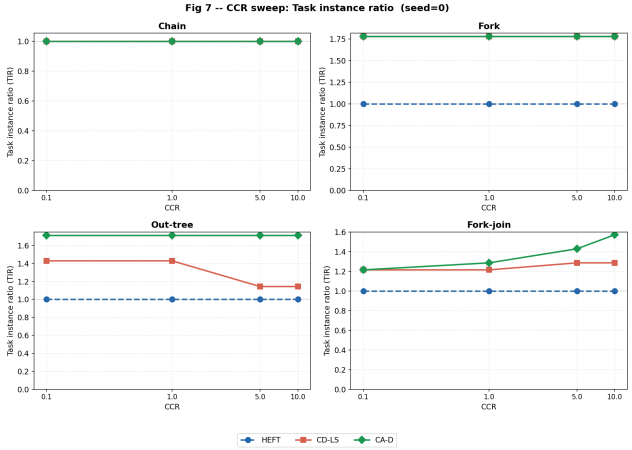


Fig. 6. Task instance ratio (TIR) vs. CCR for the four DAG families. CA-D uses more instances than CD-LS due to recursive ancestor placement, but pruning keeps TIR bounded.

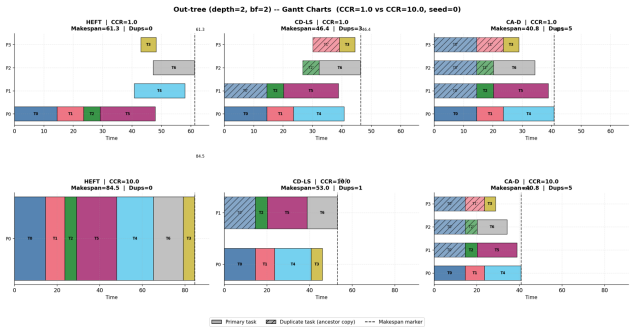


Fig. 7. Example CA-D schedule on an out-tree DAG (CCR = 1.0). Duplicated ancestor tasks on leaf processors eliminate remote communications and shorten the critical path compared to HEFT.

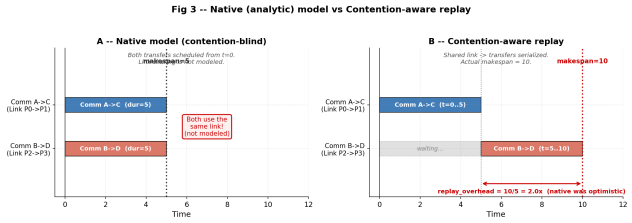


Fig. 8. Native vs. replayed makespan timing diagram. HEFT and CD-LS native schedules ignore link contention; replay materializes the real communication delays. CA-D's native and replayed makespans are nearly identical (overhead ≈ 1.0).

Table IV shows the per-family replayed speedup for CD-LS and CA-D relative to HEFT. CA-D outperforms CD-LS on every family except fork, where both are equivalent because the fork topology has only one ancestor level for duplication.

C. Discussion

Fair replay necessity. HEFT wins the majority of native comparisons on random DAGs (7 of 18) because its analytic model ignores contention. Under replay, CA-D wins 11 of 18. CD-LS natively wins 49 of 81 instances in the graph family grid but only 25 under replay. These reversals confirm that

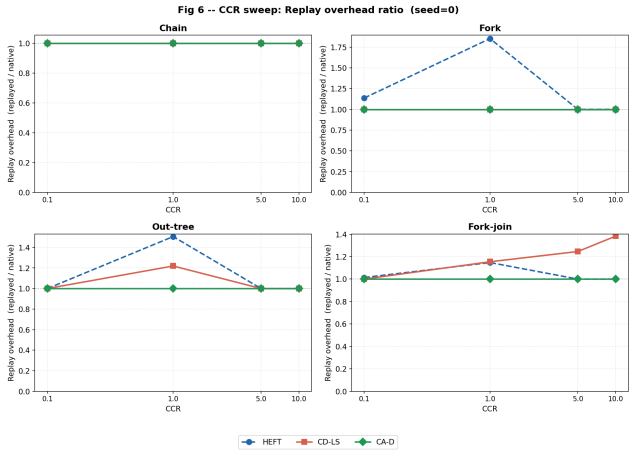


Fig. 9. Replay overhead ratio vs. CCR for each DAG family. HEFT and CD-LS overestimate performance (ratio > 1) at medium and high CCR. CA-D maintains near-unity overhead across all conditions.

TABLE IV
MEAN REPLAYED SPEEDUP VS. HEFT BY GRAPH FAMILY (EXPERIMENT B). VALUES > 1 INDICATE IMPROVEMENT OVER HEFT UNDER FAIR EVALUATION.

Family	CD-LS	CA-D
fork	**5.164**	**5.164**
join	1.117	**2.248**
out_tree	1.468	**2.053**
fork_join	1.239	**1.474**
in_tree	1.220	**1.327**
diamond	1.071	**1.541**

native makespan comparisons between contention-blind and contention-aware schedulers are systematically misleading.

CA-D vs. CD-LS. The key advantage of CA-D over CD-LS is visible at medium and high CCR on multi-level topologies (join, fork-join, diamond). CD-LS duplicates only direct parents, leaving ancestor contention unresolved. CA-D's recursive placement propagates duplicates up the dependency chain, eliminating more remote communications. On the fork topology both produce identical schedules since there is only one ancestor level.

CCR sensitivity. At CCR = 0.1, computation dominates and all schedulers achieve similar replayed speedup. The benefit of CA-D increases monotonically with CCR, reaching $2.34\times$ over HEFT at CCR = 5.0 on random DAGs.

Limitations.

- **Greedy ancestor selection:** CA-D evaluates predecessors in ascending task-id order rather than by a globally optimal critical-path criterion (Sinnen et al. Algorithm 3 [4]). This may miss beneficial duplication orderings.
- **Conservative pruning:** Phase 15B removes provably redundant duplicates but does not reroute communications; full redundant-task removal as in [4] is not implemented.
- **Runtime cost:** CA-D runtime is approximately $236\times$ HEFT on random DAGs (mean 10.4 s vs. 44 ms per instance). Fig. 10 shows the runtime breakdown. This limits practicality for large DAGs.
- **Synthetic workloads only:** No standard benchmark DAGs (e.g., STG, Pegasus, HPEC) were evaluated.
- **Excluded configurations:** 40-task random DAGs and the chain family were excluded due to CA-D runtime constraints.

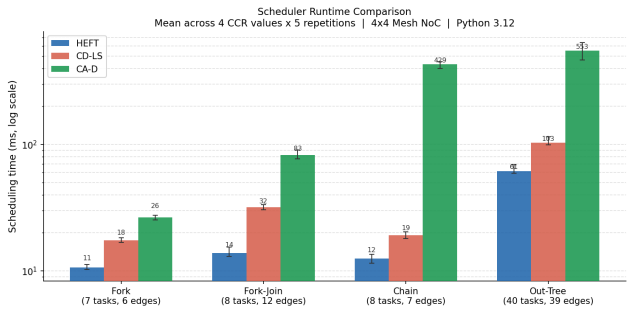


Fig. 10. Scheduler runtime (ms) grouped by CCR. CA-D's runtime grows significantly with CCR as more duplication evaluations are triggered. HEFT and CD-LS remain fast across all conditions.

Embedded system considerations. The proposed approach is an offline static scheduler suitable for design-time task mapping on NoC-based MPSoCs. The scheduler runtime overhead is a one-time design cost and does not affect execution latency. The contention model assumes whole-route atomic reservation, which is a simplification of realistic flit-level pipeline behavior.

D. Reproducibility

All experiments are reproducible from fixed seeds. The repository, environment, and main commands are summarized below.

TABLE V
REPRODUCIBILITY INFORMATION.

Item	Information
Repository	https://github.com/ogulcanuguroglu/contention-aware-noc-scheduling
Language	Python 3.10+
Dependencies	`pip install -r requirements.txt`
Tests	`pytest tests/` — 1039 unit tests
Experiment A	`python scripts/run_final_experiments.py`
Experiment B	`python scripts/run_graph_family_experiments.py`
Figures	`python scripts/generate_phase21_interpretive_figures.py`
Configuration	`configs/default.yaml`; seeds 0, 1, 2; $\alpha = 0$, $\beta = 1$, 4×4 NoC

IX. CONCLUSION AND FUTURE WORK

This work presented CA-D, a contention-aware recursive ancestor duplication heuristic for DAG scheduling on 2D mesh NoC-based multiprocessor systems. CA-D extends a standard list-scheduling framework with explicit per-link interval reservation, greedy recursive ancestor duplication (Phase 15A), and conservative redundant duplicate pruning (Phase 15B). A fair contention replay methodology enables valid makespan comparison across schedulers with different communication models.

Experiments on 72 random DAG instances and 324 structured graph family instances show that CA-D achieves a mean replayed speedup of $1.47\times$ over HEFT on random DAGs and $2.52\times$ on structured graph families, consistently outperforming the contention-blind classical duplication baseline (CD-LS) under fair evaluation. The fork topology achieves the highest benefit ($5.16\times$), while communication-light topologies such as in-tree and chain show smaller gains.

The main limitation of the current work is that CA-D uses a greedy ancestor selection order rather than the globally optimal critical-parent chain selection of Sinnen et al. [4]. Full redundant duplicate removal and rerouting are also not implemented. CA-D's scheduling runtime is substantially higher than HEFT, limiting its applicability to large DAGs in the current form.

Future work directions include:

1) Globally optimal recursive ancestor selection following Sinnen et al. Algorithm 3.
2) Full redundant duplicate removal with communication rerouting.
3) Evaluation on real application benchmark DAGs (STG, Pegasus, HPEC).
4) Extension to heterogeneous processor models.
5) CA-D runtime optimization via bounded recursion depth and candidate filtering.
6) Alpha sensitivity analysis ($\alpha > 0$) to study the effect of hop-count latency.
7) Pipelined flit-level communication model for more accurate contention representation.
8) Energy-aware duplication objective (minimize energy-delay product).
9) Evaluation on larger NoC sizes (8×8 and beyond).
10) Alternative NoC topologies (torus, fat-tree).

ACKNOWLEDGMENTS

The authors would like to thank the course instructors for their guidance, constructive feedback, and valuable insights throughout the development of this project. Their comments on problem formulation, literature positioning, and system modeling significantly improved the clarity and technical depth of this work.

The authors also acknowledge the use of publicly available research literature and tools that supported the development of the proposed methodology and evaluation framework.

Finally, the authors appreciate the discussions and feedback received during the project process, which helped refine both the technical approach and the presentation of the results.

REFERENCES

[1] D. Bozdag, U. Catalyurek, and F. Ozguner, "A task duplication based bottom-up scheduling algorithm for heterogeneous environments," in *International Conference Proceedings*, 2006.

[2] T. Hagras and J. Janecek, "A high performance, low complexity algorithm for compile-time task scheduling in heterogeneous systems," in *Proceedings of the International Parallel and Distributed Processing Symposium (IPDPS)*, 2004.

[3] S. Bansal, P. Kumar, and K. Singh, "Dealing with heterogeneity through limited duplication for scheduling precedence constrained task graphs," *Journal of Parallel and Distributed Computing*, vol. 65, pp. 479–491, 2005.

[4] O. Sinnen, A. To, and M. Kaur, "Contention-aware scheduling with task duplication," *Journal of Parallel and Distributed Computing*, vol. 71, pp. 77–86, 2011.

[5] X. Tang, K. Li, G. Liao, and R. Li, "List scheduling with duplication for heterogeneous computing systems," *Journal of Parallel and Distributed Computing*, vol. 70, pp. 323–329, 2010.

[6] Q. Tang, S.-F. Wu, J.-W. Shi, and J.-B. Wei, "Optimization of duplication-based schedules on network-on-chip based multi-processor system-on-chips," *IEEE Transactions on Parallel and Distributed Systems*, vol. 28, no. 3, pp. 826–839, 2017.

[7] S. Bansal, P. Kumar, and K. Singh, "An improved duplication strategy for scheduling precedence constrained graphs in multiprocessor systems," *IEEE Transactions on Parallel and Distributed Systems*, vol. 14, no. 6, pp. 533–544, 2003.

[8] J. Mei, K. Li, and K. Li, "A resource-aware scheduling algorithm with reduced task duplication on heterogeneous computing systems," *The Journal of Supercomputing*, vol. 68, pp. 1347–1377, 2014.

[9] A. Liang and Y. Pang, "A novel, energy-aware task duplication-based scheduling algorithm of parallel tasks on clusters," *Mathematical and Computational Applications*, vol. 22, no. 1, 2017.